1. Project Planning & Management

1.1. Project Proposal

1.1.1. Project Overview

WorkFlow is a web-based project and task management system designed to help small and medium-sized companies organize projects, tasks, employees, leave requests, and announcements in one centralized platform.

1.1.2. Project Objectives

The main objectives of WorkFlow are:

- Organize projects and tasks efficiently.
- Improve coordination between employees and managers.
- Track task progress and project status.
- Manage employee leave requests and company announcements.

1.1.3. Project Scope

The proposed system includes three user roles: Employee, Manager, and Admin. Its planned features include authentication, project and task management, employee management, leave requests, announcements, and a dashboard.

1.1.4. Target Users

The main users of WorkFlow are:

- **Employees:** View assigned tasks and projects, update task progress, read announcements, and submit leave requests.

- **Managers:** Create and manage projects, assign tasks, monitor progress, and review leave requests.
- **Administrators:** Manage employee information and oversee the system's main activities.

1.1.5. Proposed Technology Stack

The proposed technology stack consists of React, TypeScript, and Tailwind CSS for the frontend; Node.js and Express for the backend; and MongoDB for the database. Git and GitHub will be used for version control and collaboration, while Docker is planned for containerization.

1.1.6. Expected Outcome

The expected outcome is a functional, responsive, and user-friendly web application that helps organize projects and tasks, manage employee-related activities, and improve work coordination within a company.

1.2. Project Plan

The project will be developed through the following proposed phases:

1. Project Planning and Requirements Gathering
2. UI/UX Design
3. Frontend Development
4. Backend Development
5. Database Design and Integration
6. System Integration and Testing
7. Final Presentation and Documentation

Final Deadline: 4 December 2026

The detailed timeline, intermediate milestones, and task deadlines will be confirmed by the team. A Gantt chart should be prepared after these dates are agreed upon.

1.3. Task Assignment & Roles

The project will be developed by a team of five members. Work will be divided into five main areas according to task complexity and the team's agreed assignment plan.

Each member will be responsible for their assigned tasks and will collaborate with the other members during integration, testing, and documentation. All team members are expected to understand the overall system, its main features, and how its components work together.

The final allocation of responsibilities will be confirmed by the team.

1.4. Risk Assessment & Mitigation Plan

1.4.1. Potential Risks and Mitigation Strategies

Risk	Potential Impact	Mitigation Strategy
Unclear or changing requirements	Rework and delays during development.	Document the agreed requirements and review changes as a team before implementation.
Integration issues	Frontend, backend, and database components may not work together as expected.	Agree on API structures and data formats, then test integration regularly.
Uneven workload	Some tasks may take longer than expected or require additional support.	Review progress together and redistribute tasks when necessary.

Technical difficulties	Development may be delayed by unfamiliar technologies or implementation problems.	Research issues, share knowledge, and test solutions in small steps.
Bugs and unexpected errors	Some features may not work correctly.	Test features regularly and fix identified issues before final delivery.
Schedule delays	Project milestones or the final submission may be missed.	Track progress against the official timeline and identify delays early.
Version control conflicts	Team members may overwrite or conflict with each other's changes.	Use Git and GitHub consistently, communicate before integrating changes, and review code changes.
Incomplete documentation	Required project documents or reports may be missing.	Maintain documentation throughout development and review submission requirements before delivery.

1.4.2. Risk Monitoring

The team will review project progress and potential risks collaboratively. When a risk is identified, the team will discuss its impact, agree on a mitigation action, and monitor whether the action resolves the issue.

This plan may be updated as the project requirements and development process become clearer.

1.5. Key Performance Indicators (KPIs)

1.5.1. Purpose

The Key Performance Indicators (KPIs) will help the team evaluate the quality, functionality, usability, and delivery progress of the WorkFlow application.

1.5.2. Proposed KPIs

KPI	What It Measures	How It Will Be Evaluated
Feature Completion	Whether the agreed project features have been implemented.	Compare completed features with the approved requirements.
Functional Test Pass Rate	The proportion of executed functional tests that pass.	Calculate the percentage of passed tests out of all executed tests.
Defect Resolution	Whether identified software defects are being addressed.	Track reported, resolved, and outstanding defects.
Response Time	How quickly the application responds to user actions or requests.	Measure response times for selected operations under defined test conditions.
Interface Responsiveness	Whether the interface works across the target screen sizes.	Test the interface on agreed desktop, tablet, and mobile viewport sizes.
Requirement Coverage	Whether the implemented application covers the agreed requirements.	Review each requirement against its implementation and test evidence.
Milestone Completion	Whether project deliverables are completed according to the official schedule.	Compare actual progress with the confirmed milestone dates.
Documentation Completeness	Whether all required project documents and reports are prepared.	Review the documentation against the official submission checklist.

1.5.3. KPI Targets

Numerical targets will be defined by the team after confirming the project requirements, testing approach, and official delivery criteria. No numerical performance targets are assigned at this stage.

1.5.4. Monitoring and Review

The team will review these indicators during development and testing. The results will help identify incomplete features, quality issues, and schedule risks before final submission.

2. Literature Review

2.1. Introduction

Project and task management systems help organizations organize work, assign responsibilities, track progress, and improve collaboration among team members. Before developing WorkFlow, it is useful to review existing solutions, understand their main features and limitations, and identify opportunities to develop a system that meets the project's intended requirements.

This review examines three existing platforms: Asana, Trello, and Jira. They were selected because they provide project or task management capabilities that are relevant to the WorkFlow project.

2.2. Existing Solutions

2.2.1. Asana

Overview:

Asana is a project and work management platform that enables teams to organize projects, assign tasks, set due dates, monitor progress, and visualize work using different views, including lists, boards, calendars, timelines, and Gantt charts.

Advantages:

- Supports organizing projects and tasks in a centralized workspace.
- Allows tasks to have assigned owners and due dates.
- Provides multiple views for tracking work and project progress.

- Includes reporting and workload management capabilities in applicable plans.

Disadvantages and Limitations:

- Some advanced capabilities, such as additional reporting and workload management features, depend on the subscription plan.
- Its broad range of features may be more than a small team needs for a simple project.
- The platform's documented project management features do not, by themselves, establish that it meets every specific employee-management and leave-request requirement planned for WorkFlow.

Relevance to WorkFlow:

Asana demonstrates the value of centralized project management, task ownership, and progress tracking. WorkFlow can use these ideas when designing its project and task modules and dashboard.

Official source: https://asana.com/features/project-management

2.2.2. Trello

Overview:

Trello is a visual work management platform that organizes work using boards, lists, and cards. Teams can use cards to represent tasks, assign members, set due dates, and organize work through different stages.

Advantages:

- Provides a visual way to organize and track tasks.
- Supports task assignments and due dates.
- Offers automation and customization features.

- Provides a free plan with core task and board management capabilities, subject to its plan limits.

Disadvantages and Limitations:

- Its board-and-card approach may require additional configuration or complementary tools to represent more complex project structures.
- Some advanced views, administrative controls, and customization features require paid plans.
- The basic task-board model does not automatically provide all the employee-management, leave-request, and role-specific workflows planned for WorkFlow.

Relevance to WorkFlow:

Trello demonstrates how visual task statuses can help users understand work progress. WorkFlow can apply a similar concept by allowing tasks to move through statuses such as To Do, In Progress, Review, and Completed.

Official sources:

- https://trello.com/en/magic
- https://trello.com/en/pricing

2.2.3. Jira

Overview:

Jira is a project management and issue-tracking platform that supports organizing work, managing tasks, defining workflows, and monitoring progress through reports and dashboards.

Advantages:

- Supports structured workflows for tracking work items.

- Provides multiple project views, including boards, lists, timelines, and calendars.
- Offers reports and dashboards to help teams monitor progress.
- Includes automation capabilities to reduce repetitive work.

Disadvantages and Limitations:

- Some administration, permission-management, and security capabilities are limited on the free plan.
- Its extensive workflow and project configuration options may require additional time for teams that need only a straightforward task management system.
- Its standard project-management capabilities do not automatically satisfy every company-specific employee-management and leave-request requirement planned for WorkFlow.

Relevance to WorkFlow:

Jira highlights the importance of defining clear task workflows, monitoring progress, and using dashboards. These concepts can inform WorkFlow's task-status process and project overview dashboard.

Official sources:

- https://www.atlassian.com/software/jira/jira/pricing
- https://support.atlassian.com/jira-cloud-administration/docs/what-is-the-free-jira-cloud-plan/

2.3. Comparison with WorkFlow

WorkFlow is planned as a web-based project and task management system for a small or medium-sized company. Its intended features include project management, task management, employee management, leave requests,

company announcements, and a dashboard. It will use React, TypeScript, Tailwind CSS, Node.js, Express, and MongoDB.

The following comparison is based on the published features of the existing platforms and the currently planned scope of WorkFlow.

Feature or Aspect	Asana	Trello	Jira	WorkFlow (Planned)
Project management	Supported	Supported through boards and cards	Supported	Planned
Task assignment and tracking	Supported	Supported	Supported	Planned
Visual task workflow	Multiple project views	Boards, lists, and cards	Configurable workflows and boards	Planned task statuses
Progress monitoring	Project views and reporting	Visual board tracking	Reports and dashboards	Planned dashboard
Employee management	General team-management capabilities	Not established as a complete employee-management module by the reviewed sources	Not established as a complete employee-management module by the reviewed sources	Planned
Leave requests	Not verified as a built-in feature matching WorkFlow's intended requirements	Not verified as a built-in feature matching WorkFlow's intended requirements	Not verified as a built-in feature matching WorkFlow's intended requirements	Planned
Company announcements	Not verified as matching WorkFlow's intended announcements module	Not verified as matching WorkFlow's intended announcements module	Not verified as matching WorkFlow's intended announcements module	Planned

Technology stack	Existing hosted platform	Existing hosted platform	Existing hosted platform	React, TypeScript, Tailwind CSS, Node.js, Express, and MongoDB

The table describes the features relevant to this review. "Not verified" means that the reviewed sources did not establish a feature that matches WorkFlow's planned module; it does not necessarily mean that the platform cannot support similar functionality through configuration or integrations.

2.4. Findings and Opportunities for Improvement

The review identifies several useful ideas for WorkFlow:

1. **Centralized project and task management:** Following the general approach used by Asana, WorkFlow should provide a central place for users to access projects and their associated tasks.
2. **Clear visual task statuses:** Inspired by Trello's visual organization, WorkFlow can make task progress easy to understand through clearly defined statuses.
3. **Structured workflows and progress reporting:** Jira demonstrates the value of structured workflows, reports, and dashboards. WorkFlow can apply these principles to its planned task-management and dashboard features.
4. **Role-specific functionality:** WorkFlow is intended to provide different experiences for employees, managers, and administrators. The system should ensure that each role can access the functions relevant to its responsibilities.

5. **Company-related features:** Employee management, leave requests, and announcements are part of WorkFlow's planned scope. These features should be developed according to the team's confirmed requirements rather than assumed to be unique or unavailable in all existing platforms.
6. **Manageable project scope:** Since WorkFlow is being developed by a five-member team, the team should prioritize the agreed core features and verify their functionality before adding optional enhancements.

2.5. Conclusion

The review of Asana, Trello, and Jira shows several approaches to organizing projects, tracking tasks, defining workflows, and monitoring progress. These approaches provide useful references for WorkFlow's design.

WorkFlow's planned scope combines project and task management with employee management, leave requests, announcements, and a dashboard for different user roles. The project will use React, TypeScript, Tailwind CSS, Node.js, Express, and MongoDB.

The comparison will help the team refine its requirements, prioritize features, and make informed design decisions. WorkFlow's final capabilities and differences will need to be verified against the features actually implemented and tested.

2.6. References

1. Asana. "Project Management." https://asana.com/features/project-management
2. Trello. "Teamwork That Feels Like Magic." https://trello.com/en/magic
3. Trello. "Trello Pricing." https://trello.com/en/pricing
4. Atlassian. "Jira Pricing." https://www.atlassian.com/software/jira/jira/pricing

5. Atlassian Support. "What Is the Free Jira Cloud Plan?" https://support.atlassian.com/jira-cloud-administration/docs/what-is-the-free-jira-cloud-plan/

3. Requirements Gathering

3.1. Stakeholder Analysis

3.1.1. Overview

Stakeholder analysis identifies the people or groups who interact with the WorkFlow system, use its features, or have an interest in its success. Understanding their needs helps the team define the system requirements and develop appropriate functionality.

3.1.2. Key Stakeholders and Their Needs

1. Employees

Employees are the primary users who need to access their assigned work and company-related information.

Their needs include:

- Viewing assigned projects and tasks.
- Checking task statuses and updating progress.
- Submitting leave requests and viewing their status.
- Reading company announcements.
- Accessing relevant information through the system dashboard.

2. Managers

Managers need to organize work, coordinate employees, and monitor project progress.

Their needs include:

- Creating and managing projects.
- Creating tasks and assigning them to employees.
- Monitoring task statuses and project progress.
- Reviewing employee leave requests.
- Publishing company announcements, if this permission is included in the final requirements.
- Viewing relevant project and task information through the dashboard.

3. Administrators

Administrators are responsible for managing employee information and overseeing the system's main administrative functions.

Their needs include:

- Managing employee accounts and information.
- Managing access permissions according to user roles.
- Maintaining organized system information.
- Accessing relevant administrative information through the dashboard.

4. Project Team

The project team consists of the five members responsible for planning, developing, testing, and documenting WorkFlow.

Their needs include:

- Understanding the project requirements.
- Coordinating frontend, backend, and database development.
- Testing system functionality and identifying defects.

- Maintaining project documentation and preparing the final presentation.

5. Lecturer or Project Evaluator

The lecturer or evaluator reviews the project's deliverables and assesses its implementation and presentation according to the training program's requirements.

Their needs include:

- Access to the required project documentation.
- A clear demonstration of the implemented features.
- Evidence of testing and project completion.

The lecturer or evaluator is a project stakeholder, but is not necessarily a user of the WorkFlow application itself.

3.2. User Stories & Use Cases

3.2.1. User Stories

User stories describe system functionality from the perspective of the person who needs it.

The following user stories represent the initial planned requirements for WorkFlow.

Employee User Stories

- As an employee, I want to view my assigned tasks so that I can understand my responsibilities.
- As an employee, I want to update the status of my tasks so that my progress can be tracked.

- As an employee, I want to submit a leave request so that my manager can review it.
- As an employee, I want to check the status of my leave request so that I know whether it has been approved or rejected.
- As an employee, I want to read company announcements so that I can stay informed about important updates.

Manager User Stories

- As a manager, I want to create projects so that company work can be organized.
- As a manager, I want to create and assign tasks so that employees know what they need to complete.
- As a manager, I want to monitor task and project progress so that I can identify delays.
- As a manager, I want to review leave requests so that I can approve or reject them.
- As a manager, I want to view project information on a dashboard so that I can monitor ongoing work.

Administrator User Stories

- As an administrator, I want to manage employee information so that company records remain organized.
- As an administrator, I want to manage user access according to assigned roles so that users can access appropriate system functions.

3.2.2. Use Cases

Use Case 1: Employee Updates a Task

- **Primary Actor:** Employee
- **Precondition:** The employee is logged in and has an assigned task.
- **Main Flow:**

    - The employee opens the task list.
    - The employee selects an assigned task.
    - The employee reviews the task details.
    - The employee updates the task status.
    - The system saves the change and displays the updated status.
- **Expected Result:** The task status is updated and the new status is available for progress tracking.

Use Case 2: Employee Submits a Leave Request

- **Primary Actor:** Employee
- **Precondition:** The employee is logged in.
- **Main Flow:**
    - The employee opens the leave-request section.
    - The employee enters the required leave details.
    - The employee submits the request.
    - The system records the request.
    - The request becomes available for review by an authorized manager.
- **Expected Result:** The leave request is recorded and can be reviewed.

Use Case 3: Manager Assigns a Task

- **Primary Actor:** Manager
- **Precondition:** The manager is logged in and has permission to manage the relevant project.
- **Main Flow:**
    - The manager opens a project.
    - The manager creates a task and enters its details.
    - The manager selects an employee to assign the task to.
    - The manager saves the task.
    - The system records the task and its assignment.

- **Expected Result:** The task is associated with the selected employee and project.

Use Case 4: Manager Reviews a Leave Request

- **Primary Actor:** Manager
- **Precondition:** The manager is logged in and has permission to review leave requests.
- **Main Flow:**
    - The manager opens the leave-request section.
    - The manager selects a pending request.
    - The manager reviews the submitted details.
    - The manager approves or rejects the request.
    - The system records the decision and updates the request status.
- **Expected Result:** The request status reflects the manager's decision.

Use Case 5: Administrator Manages an Employee Account

- **Primary Actor:** Administrator
- **Precondition:** The administrator is logged in and has the required permissions.
- **Main Flow:**
    - The administrator opens the employee-management section.
    - The administrator selects an employee record.
    - The administrator reviews or updates the permitted information.
    - The administrator saves the changes.
    - The system records the updated information.
- **Expected Result:** The employee information is updated according to the administrator's permissions.

3.3. Functional Requirements

3.3.1. Overview

Functional requirements describe the features and operations that the WorkFlow system is expected to provide.

The following requirements are proposed based on the project's currently planned scope. They should be reviewed and confirmed by the team before implementation.

3.3.2. Authentication and User Roles

- **FR-01:** The system shall allow authorized users to log in.
- **FR-02:** The system shall distinguish between Employee, Manager, and Administrator roles.
- **FR-03:** The system shall restrict access to system functions according to the user's permissions.

3.3.3. Project Management

- **FR-04:** The system shall allow authorized users to create and manage projects.
- **FR-05:** The system shall allow users with appropriate permissions to view relevant project information.
- **FR-06:** The system shall provide access to project progress information.

3.3.4. Task Management

- **FR-07:** The system shall allow authorized users to create tasks and assign them to employees.
- **FR-08:** The system shall allow users with appropriate permissions to view task details.

- **FR-09:** The system shall allow employees to update the status of their assigned tasks.
- **FR-10:** The system shall support task statuses such as To Do, In Progress, Review, and Completed, subject to final team approval.

3.3.5. Employee Management

- **FR-11:** The system shall allow administrators to manage employee information.
- **FR-12:** The system shall provide employee access to their relevant profile and work information.

3.3.6. Leave Requests

- **FR-13:** The system shall allow employees to submit leave requests.
- **FR-14:** The system shall allow authorized managers to review leave requests.
- **FR-15:** The system shall record the decision and current status of each leave request.
- **FR-16:** The system shall allow employees to view the status of their submitted requests.

3.3.7. Announcements

- **FR-17:** The system shall allow authorized users to publish company announcements.
- **FR-18:** The system shall allow employees to view relevant published announcements.

3.3.8. Dashboard

- **FR-19:** The system shall provide a dashboard displaying relevant project, task, or administrative information according to the user's role.

- **FR-20:** The information displayed on the dashboard shall reflect the data available in the system.

3.4. Non-functional Requirements

3.4.1. Overview

Non-functional requirements describe the expected quality and operational characteristics of the WorkFlow system, including performance, security, usability, and reliability.

The following are proposed quality requirements. Numerical targets will be defined after the team confirms the evaluation criteria and testing conditions.

3.4.2. Performance

- **NFR-01:** The system should respond to user actions within a reasonable time under the expected usage conditions.
- **NFR-02:** The system should retrieve and display project, task, and employee information efficiently.
- **NFR-03:** Performance should be evaluated through testing under defined conditions.

3.4.3. Security

- **NFR-04:** The system shall protect restricted functions and information from unauthorized access.
- **NFR-05:** The system shall validate permissions on the server when users attempt to access protected resources or perform restricted actions.
- **NFR-06:** The system shall handle authentication and user credentials securely.

- **NFR-07:** The system shall validate user input and handle errors without unnecessarily exposing sensitive information.

3.4.4. Usability

- **NFR-08:** The user interface should be clear and easy to understand.
- **NFR-09:** The system should provide navigation appropriate to the user's role.
- **NFR-10:** The interface should adapt to the target desktop, tablet, and mobile screen sizes.

3.4.5. Reliability

- **NFR-11:** The system should handle expected user operations and invalid input without unexpected application failures.
- **NFR-12:** The system should provide clear feedback when an operation succeeds or fails.
- **NFR-13:** The team should test the main application features and fix identified defects before final delivery.

3.4.6. Maintainability

- **NFR-14:** The codebase should be organized into understandable and maintainable components.
- **NFR-15:** The frontend, backend, and database interactions should be structured to support testing and future modifications.
- **NFR-16:** Git and GitHub should be used to track code changes and support team collaboration.

3.4.7. Compatibility

- **NFR-17:** The web application should work correctly in the agreed target web browsers.

- **NFR-18:** The interface should remain usable across the agreed target screen sizes.